

Python E-Course

Module 14 — Capstone Projects

Detailed Notes

Module Overview

Level: Applied

Duration: ~2 weeks

Prerequisites: Modules 1-13

What This Module Is

Two substantial projects that pull together everything you've learned. Build them end-to-end: spec → design → code → test → ship. The work here is meant to feel like real software development.

You'll deliver:

- A complete, polished portfolio project (your choice from two tracks).
- A second smaller project that exercises different parts of the curriculum.
- A short README for each, suitable to show recruiters.

Lessons

These lessons are scaffolding for the capstone work — less new syntax, more how to build:

#	Lesson	Duration
1	How to Plan a Real Project	30 min
2	Project Structure for the Capstone	30 min
3	Testing with unittest and pytest	40 min
4	Documenting and Shipping	30 min
5	Next Steps After This Course	25 min

The Capstone Projects

Pick one from each tier:

Tier A — Main project (~12-20 hours)

- A1: Personal Finance Tracker — multi-account, categories, reports, charts via terminal or simple matplotlib, persistence in SQLite, optional CSV import.
- A2: News Aggregator — fetch from 1-2 public news APIs, store in SQLite, filter/search, simple CLI dashboard, optional weekly email.
- A3: Habit Tracker — daily check-ins, streaks, weekly report, CLI plus JSON/SQLite persistence.

Tier B — Smaller project (~4-8 hours)

- B1: Markdown TOC Generator — read a Markdown file, build a table of contents from headings.
- B2: Bulk File Renamer — rename files in a folder by a pattern, with dry-run support.

- B3: Personal Wikipedia Bot — accept a query, hit Wikipedia API, return a 2-paragraph summary.

Assessment

- Exercises — small warmup tasks per scaffolding lesson
- Quiz — short knowledge check
- Assignment — the Tier A capstone spec
- Project — the Tier B mini-capstone spec

What's Next

After this module: see [career/career-guide.md](#) and [career/portfolio-ideas.md](#). Run the final certification test.

Lesson 1: How to Plan a Real Project

Module: 14 — Capstone

Duration: 30 minutes

Difficulty: Applied

Learning Objectives

- Translate an idea into a one-page spec.
- Break a spec into a milestone list.
- Identify the smallest first version that's actually usable.

Prerequisites

- Modules 1-13.

1. Why This Matters

The single biggest reason hobby projects die is that the author tries to build everything at once. A short spec + a tiny first version + iterative additions consistently beats a giant plan. This lesson teaches the rhythm.

2. Concept

2.1 The one-page spec

Write this before opening an editor:

- What is it? One sentence.
- Who is it for? Even if just you.
- What does v1 do? Bulleted, ≤ 5 items.
- What does v1 explicitly NOT do? Equally important.
- What does success look like? A concrete sentence: "I can ..."

Example for the Finance Tracker:

What: A CLI tool to record and categorize personal expenses.
For: Me, to replace the spreadsheet I keep losing.
v1 does: add expense, list, monthly summary, export CSV.
v1 not: multi-user, web UI, charts, recurring expenses, currency conversion.
Success: I can record a week of expenses in under 30s per entry,
and the monthly summary shows top categories accurately.

2.2 Milestones

Break v1 into 3-5 milestones, each ~half a day:

M1. SQLite schema + connect + init.
M2. add + list commands working from CLI.
M3. summary command with grouping.

M4. CSV export.
M5. tests + README + polish.

You'll know you're done with a milestone when you've used it.

2.3 Walking skeleton first

Build the smallest thing that touches every layer end-to-end:

- One CLI command.
- One DB write.
- One DB read.
- One printed report.

Once that works, add features. This catches integration problems early.

2.4 "Just enough" architecture

For a capstone-sized project, a single package with 4-7 modules is plenty:

```
finance/  
├─ __init__.py  
├─ __main__.py      # entry point  
├─ cli.py           # argparse + dispatch  
├─ db.py            # connect, schema, queries  
├─ report.py        # formatting, summaries  
└─ errors.py
```

Don't add layers (services/, repositories/, adapters/) until you feel pain without them.

2.5 Track your work

A TODO.md (or a list in your editor) is enough. Cross things off; write tomorrow's first task before stopping today.

2.6 Time budget

Decide up front how long v1 should take. Real Tier-A capstone: ~12-20 hours. If you're at 25 hours and still 60% done, cut features — don't extend the schedule.

3. Code Examples

(This lesson is process, not code. Templates instead.)

Template 1: Spec

```
# <Project Name>  
  
## What  
<one sentence>  
  
## For  
<one sentence>  
  
## v1 features
```

```

- ...
- ...

## v1 explicit non-goals
- ...
- ...

## Success criteria
- ...

```

Template 2: Milestone list

```

- [ ] M1: schema + init
- [ ] M2: add + list
- [ ] M3: summary
- [ ] M4: export
- [ ] M5: README + tests

```

Template 3: Daily restart note

Yesterday: finished M2 (add + list).

Today: start summary – group by category, sort desc.

Blocker: none.

4. Common Mistakes

Mistake	What Happens	Fix
Endless scope creep	Project never ships	Write non-goals; cut features
Architecting before coding	Stuck in design	Walking skeleton first
Skipping the spec	Scattered, wandering work	Spend 20 minutes writing one
Adding layers preemptively	Drown in indirection	Add complexity when it earns its place

5. Practice Tasks

- Pick one of the Tier-A capstones from the README. Write its one-page spec.
- Break it into 4-6 milestones.
- Write the first task of milestone 1 in concrete terms.

6. Key Takeaways

- Spec first, code second.
- Walking skeleton end-to-end before adding features.
- Non-goals are as important as goals.
- Tiny milestones, daily progress.

7. What's Next

A look at project structure specific to the capstone.

Lesson 2: Project Structure for the Capstone

Module: 14 — Capstone

Duration: 30 minutes

Difficulty: Applied

Learning Objectives

- Lay out a small-to-mid Python project that's easy to navigate.
- Separate I/O, business logic, and presentation.
- Set up pyproject.toml, README.md, .gitignore, and a tests/ folder.

Prerequisites

- Module 10 Lesson 5

1. Why This Matters

The structure you start with shapes how easy the project is to read, change, and demo. The layout below is what a junior dev should reach for by default — small enough not to be over-engineered, large enough to support real growth.

2. Concept

2.1 The default layout

```
finance/
├── .gitignore
├── README.md
├── pyproject.toml
├── requirements.txt          (optional, for app-style projects)
├── .env.example             (if you use env vars)
├── finance/                 <- the package
│   ├── __init__.py
│   ├── __main__.py
│   ├── cli.py
│   ├── db.py
│   ├── report.py
│   └── errors.py
└── tests/
    ├── __init__.py
    ├── test_db.py
    └── test_report.py
```

You can run it with `python -m finance` after installing locally: `pip install -e ..`

2.2 Separate I/O, logic, presentation

A solid rule: business logic shouldn't know how it's called or how it's displayed.

- `db.py` — talks to the database.
- `report.py` — pure functions: given data, produce summaries.

- cli.py — parses arguments, dispatches, formats output.

This makes report.py testable without spinning up a DB or CLI.

2.3 __main__.py for python -m

```
# finance/__main__.py
from .cli import run

if __name__ == "__main__":
    run()
```

2.4 Errors module

```
# finance/errors.py
class FinanceError(Exception):
    """Base."""

class NotFound(FinanceError):
    pass
```

Importable from from finance.errors import FinanceError.

2.5 pyproject.toml skeleton

```
[project]
name = "finance"
version = "0.1.0"
description = "Personal finance tracker (capstone)."
```

```
requires-python = ">=3.10"
dependencies = []          # add 3rd-party deps here
```

```
[build-system]
requires = ["setuptools>=61"]
build-backend = "setuptools.build_meta"
```

2.6 README.md skeleton

Finance

Track personal expenses from the command line.

Install

```
python -m venv .venv source .venv/bin/activate pip install -e .
```

Use

```
python -m finance add 450 food --note "lunch" python -m finance list python -m finance
summary
```

Develop

```
pip install -r requirements-dev.txt pytest
```


2.7 .gitignore

```
.env/  
__pycache__/  
*.pyc  
.env  
.coverage  
.pytest_cache/  
finance.db
```

(Add *.db if your project uses SQLite and you don't want sample data committed.)

3. Code Examples

Example 1: Tiny end-to-end skeleton

finance/__init__.py:

```
__version__ = "0.1.0"
```

finance/db.py:

```
import sqlite3  
from pathlib import Path  
  
DEFAULT_PATH = Path("finance.db")  
  
def connect(path=DEFAULT_PATH):  
    conn = sqlite3.connect(path)  
    conn.row_factory = sqlite3.Row  
    return conn  
  
def init_schema(conn):  
    conn.executescript("""  
    CREATE TABLE IF NOT EXISTS expenses (  
        id INTEGER PRIMARY KEY,  
        amount REAL NOT NULL,  
        category TEXT NOT NULL,  
        note TEXT,  
        spent_at TEXT NOT NULL  
    );  
    """)
```

finance/report.py:

```
from collections import defaultdict  
  
def by_category(rows):  
    totals = defaultdict(float)  
    for r in rows:  
        totals[r["category"]] += r["amount"]  
    return sorted(totals.items(), key=lambda kv: kv[1], reverse=True)
```

finance/cli.py:

```
import argparse
from .db import connect, init_schema
from .report import by_category

def run(argv=None):
    parser = argparse.ArgumentParser(prog="finance")
    sub = parser.add_subparsers(dest="cmd", required=True)
    sub.add_parser("summary")
    add = sub.add_parser("add")
    add.add_argument("amount", type=float)
    add.add_argument("category")
    add.add_argument("--note", default="")
    args = parser.parse_args(argv)

    conn = connect()
    init_schema(conn)

    if args.cmd == "summary":
        rows = conn.execute("SELECT * FROM expenses").fetchall()
        for cat, total in by_category(rows):
            print(f"{cat:<12} {total:>10.2f}")
    elif args.cmd == "add":
        from datetime import date
        with conn:
            conn.execute(
                "INSERT INTO expenses (amount, category, note, spent_at) VALUES
(?)",
                (args.amount, args.category, args.note, date.today().isoformat()),
            )
            print("added.")
```

finance/__main__.py:

```
from .cli import run
if __name__ == "__main__":
    run()
```

Run:

```
$ pip install -e .
$ python -m finance add 450 food --note "lunch"
added.
$ python -m finance summary
food                450.00
```

4. Common Mistakes

Mistake	What Happens	Fix
Mixing I/O into business logic	Hard to test	Keep report.py pure
cli.py doing everything	Becomes a 500-line god module	Delegate to small functions

from finance import *	Confuses tools	Explicit imports only
No pyproject.toml	Can't pip install -e .	Add it

5. Practice Tasks

- Lay out the structure above for your Tier-A capstone choice.
- Add a working `__main__.py` and `cli.py` with one subcommand.
- Verify `python -m yourproject --help` works.

6. Key Takeaways

- Small package + `__main__.py` + clear modules.
- Separate I/O, logic, presentation.
- `pyproject.toml` + `README` + `.gitignore` are non-optional even for personal projects.

7. What's Next

Testing — give your project the same safety net you've seen in larger codebases.

Lesson 3: Testing with unittest and pytest

Module: 14 — Capstone

Duration: 40 minutes

Difficulty: Applied

Learning Objectives

- Write basic tests with unittest.
- Write tests with pytest and run them.
- Use fixtures for setup, including a temporary database.

Prerequisites

- Modules 1-13. Especially Module 4 (functions) and Module 13 (databases).

1. Why This Matters

Tests are the difference between code that works once and code that keeps working as you change it. For your capstone, even ~10 tests on the core logic will pay back instantly. Once you've written tests, you stop fearing to refactor.

2. Concept

2.1 unittest (stdlib)

```
# tests/test_math.py
import unittest

def double(x):
    return x * 2

class TestDouble(unittest.TestCase):
    def test_positive(self):
        self.assertEqual(double(3), 6)
    def test_zero(self):
        self.assertEqual(double(0), 0)

if __name__ == "__main__":
    unittest.main()
```

Run:

```
python -m unittest discover tests
```

2.2 pytest (third-party — preferred)

Install:

```
pip install pytest
```

Same idea, no class needed:

```
# tests/test_math.py
def double(x):
    return x * 2

def test_positive():
    assert double(3) == 6

def test_zero():
    assert double(0) == 0
```

Run:

```
pytest
```

pytest auto-discovers test_.py files and test_ functions. Output is pretty and assertion failures explain themselves.

2.3 Common assertions

```
assert x == y
assert x > 0
assert "foo" in result
assert isinstance(obj, MyClass)
```

For exceptions:

```
import pytest
def test_raises():
    with pytest.raises(ValueError):
        int("abc")
```

For floats:

```
import math
assert math.isclose(value, 0.3, rel_tol=1e-9)
```

2.4 Fixtures

A fixture is a setup function whose return value is injected into tests:

```
import pytest, sqlite3

@pytest.fixture
def conn():
    c = sqlite3.connect(":memory:")
    c.execute("CREATE TABLE t (id INTEGER PRIMARY KEY, x INTEGER)")
    yield c
    c.close()

def test_insert(conn):
    conn.execute("INSERT INTO t (x) VALUES (?)", (5,))
    assert conn.execute("SELECT x FROM t").fetchone()[0] == 5
```

The yield separates setup from teardown.

2.5 Parametrize

Run the same test against multiple inputs:

```
import pytest

@pytest.mark.parametrize("a,b,expected", [
    (1, 2, 3),
    (0, 0, 0),
    (-1, 1, 0),
])
def test_add(a, b, expected):
    assert a + b == expected
```

2.6 What to test in your capstone

Focus on pure logic in report.py-style modules: aggregations, filters, calculations. They're easy to test and most likely to break when you refactor.

Don't try to test every CLI path — argparse + print is hard to test and rarely the source of bugs.

2.7 Folder layout

```
myproject/
├── myproject/
│   └── ...
└── tests/
    ├── __init__.py
    └── test_*.py
```

pytest finds tests automatically; no extra config needed.

3. Code Examples

Example 1: Test a pure function

report.py:

```
def by_category(rows):
    totals = {}
    for r in rows:
        totals[r["category"]] = totals.get(r["category"], 0) + r["amount"]
    return sorted(totals.items(), key=lambda kv: kv[1], reverse=True)
```

tests/test_report.py:

```
from finance.report import by_category

def test_empty():
    assert by_category([]) == []

def test_groups_and_sorts():
    rows = [
        {"category": "food", "amount": 100},
        {"category": "food", "amount": 50},
```

```

        {"category": "rent", "amount": 1000},
    ]
    assert by_category(rows) == [("rent", 1000), ("food", 150)]

```

Run pytest:

```

===== test session starts =====
collected 2 items

tests/test_report.py ..                [100%]

===== 2 passed in 0.04s =====

```

Example 2: Fixture with temp DB

tests/test_db.py:

```

import sqlite3
import pytest
from finance import db

@pytest.fixture
def conn():
    c = sqlite3.connect(":memory:")
    c.row_factory = sqlite3.Row
    db.init_schema(c)
    yield c
    c.close()

def test_insert_and_count(conn):
    conn.execute(
        "INSERT INTO expenses (amount, category, note, spent_at) VALUES
    (?, ?, ?, ?)",
        (100.0, "food", "", "2026-05-23"),
    )
    rows = conn.execute("SELECT COUNT(*) FROM expenses").fetchone()
    assert rows[0] == 1

```

Example 3: parametrize

tests/test_grade.py:

```

import pytest

def grade(score):
    return "F" if score < 40 else "C" if score < 60 else "B" if score < 80 else "A"

@pytest.mark.parametrize("score,expected", [
    (10, "F"), (45, "C"), (60, "B"), (85, "A"), (100, "A"),
])
def test_grade(score, expected):
    assert grade(score) == expected

```

Example 4: Exception

```
import pytest

def safe_div(a, b):
    if b == 0:
        raise ZeroDivisionError("nope")
    return a / b

def test_raises():
    with pytest.raises(ZeroDivisionError):
        safe_div(1, 0)
```

4. Common Mistakes

Mistake	What Happens	Fix
Testing implementation details	Tests break on refactor	Test inputs → outputs
One huge test_everything()	Hard to debug failures	Many small tests
Tests that depend on each other	Order-sensitive flakes	Each test sets up its own state
No fixture for shared setup	Repeating code	Use <code>@pytest.fixture</code>

5. Practice Tasks

- Install pytest in your capstone venv.
- Write 3 tests for any pure function you have.
- Add a parametrized test for a function with 3+ branches.

6. Key Takeaways

- pytest over unittest for new code.
- Test pure logic first; skip CLI/print.
- Fixtures isolate state; parametrize covers multiple inputs.
- ~10 well-chosen tests beat 100 reckless ones.

7. What's Next

Documenting and shipping — how to make your project look ready to be shown.

Lesson 4: Documenting and Shipping

Module: 14 — Capstone

Duration: 30 minutes

Difficulty: Applied

Learning Objectives

- Write a README.md that explains what, install, use, and develop.
- Push a project to GitHub.
- Tag a release and write changelog notes.

Prerequisites

- A capstone in progress.
- A GitHub account (free).

1. Why This Matters

A great project no one understands is dead on arrival. Shipping is its own skill: clear README, public repo, basic versioning. This is what recruiters see — invest in it.

2. Concept

2.1 README structure

```
# Project Name

One-line description.

## Demo
(GIF, screenshot, or sample output)

## Features
- bullet 1
- bullet 2

## Install
$ python -m venv .venv
$ source .venv/bin/activate
$ pip install -e .

## Use
$ python -m project ...
(more examples)

## Develop
$ pip install -r requirements-dev.txt
$ pytest

## Stack
```

```
Python 3.10+, sqlite3, requests
```

```
## License  
MIT
```

Keep it short — 80% of useful README is Install + Use.

2.2 Pictures help

For a CLI, paste actual terminal output:

```
$ python -m finance summary food      450.00 transport  1200.00
```

For anything visual, a screenshot or GIF (record with peek, licesap, screentogif) is gold.

2.3 Push to GitHub

```
git init  
git add .  
git commit -m "Initial commit"  
git branch -M main  
git remote add origin git@github.com:USERNAME/projectname.git  
git push -u origin main
```

GitHub now hosts the project. Make sure your README renders correctly on the repo page.

2.4 Tagging a release

After v1 is working:

```
git tag -a v0.1.0 -m "First release"  
git push origin v0.1.0
```

On GitHub, "Releases" → "Draft a new release" → pick the tag → write 3-5 bullet points about what's in this version.

2.5 A CHANGELOG file

Keep CHANGELOG.md:

```
# Changelog  
  
## [0.2.0] - 2026-06-10  
### Added  
- CSV export  
- Date range filters  
  
## [0.1.0] - 2026-06-01  
### Added  
- Add, list, summary commands  
- SQLite persistence
```

This is what users (and future you) read to know what changed.

2.6 LICENSE

Add a LICENSE file — MIT or Apache 2.0 is fine for personal projects. Use choosealicense.com if unsure.

2.7 Polish checklist

Before declaring done:

- [] README has Install, Use, Demo.
- [] `python -m project --help` works.
- [] At least 5 tests pass.
- [] No commented-out debug code.
- [] `.gitignore` excludes secrets, `.venv`, DB files.
- [] `pyproject.toml` has a version and description.
- [] Pushed to GitHub.
- [] Tagged v0.1.0.

2.8 What to say about it

In a portfolio: "Built a Python CLI for X. Uses SQLite for persistence, tests with pytest, packaged with `pyproject.toml`. ~600 lines, ~10 tests." Specifics beat adjectives.

3. Code Examples

Example 1: README template

(See section 2.1 — copy-paste, edit for your project.)

Example 2: Commit history that tells a story

```
$ git log --oneline
5f2a1b8 docs: README + LICENSE
6a91d2c feat: CSV export
4f1d3a8 feat: monthly summary command
3a92db1 feat: add and list commands
1e02fa8 chore: initial scaffolding
```

Good commit messages: imperative ("add CSV export") and small.

Example 3: A working CLI help screen

```
$ python -m finance --help
usage: finance [-h] {add,list,summary} ...
```

Personal expense tracker.

```
positional arguments:
  {add,list,summary}
    add                record a new expense
    list               show recent expenses
    summary            per-category totals
```

```
optional arguments:
  -h, --help            show this help message
```

When your help looks like this, the project feels professional.

4. Common Mistakes

Mistake	What Happens	Fix
README is the spec	Recruiter can't run it	Lead with Install + Use
Pushing secrets to GitHub	API key compromised	Audit + rotate + .gitignore
One giant commit	No story	Commit per logical step
Skipping LICENSE	Legal ambiguity	Add MIT

5. Practice Tasks

- Write a README for your Tier-A capstone using the template.
- Initialize git, commit, push to a fresh GitHub repo.
- Tag v0.1.0 once your v1 is working.

6. Key Takeaways

- README sells the project — keep it concrete.
- Push to GitHub; tag releases; keep a changelog.
- Polish checklist before declaring done.

7. What's Next

Final lesson: where to go after this course.

Lesson 5: Next Steps After This Course

Module: 14 — Capstone

Duration: 25 minutes

Difficulty: Applied

Learning Objectives

- Map possible specialization paths.
- Pick the right next tool depending on your goal.
- Set up a sustainable learning rhythm.

Prerequisites

- Finished or nearly-finished capstone.

1. Why This Matters

Python is wide. After this course you have a strong foundation; the next step is depth in one direction. Picking that direction is more impactful than learning ten things shallowly.

2. Concept

2.1 Specialization paths

If you like...	Go deeper in	Key tools
Web backends	Flask → FastAPI / Django	FastAPI, SQLAlchemy, Pydantic, Postgres, Docker
Data science	Pandas, plotting, ML	NumPy, pandas, matplotlib, scikit-learn, Jupyter
Machine learning	Deep learning, MLOps	PyTorch / JAX, Hugging Face, Weights & Biases
Automation	DevOps + scripting	Click, Rich, Ansible, GitHub Actions
Scraping	Web scraping	Playwright, BeautifulSoup, Scrapy
Game dev	Game programming	Pygame, Godot (Python bindings)

You only need to pick one for now. Switching later is easy because your fundamentals stay.

2.2 Read source code

Pick a small Python library you use (requests, httpx, rich) and read its source. You'll learn more from a real codebase than from another tutorial.

2.3 Build, don't just consume

Tutorials feel productive but rarely build skill. Instead:

- Pick a problem from your life. Build a tool for it.

- Reproduce something you admire — a tiny clone of a tool.
- Contribute to an open-source library (start with docs typo fixes).

A rough rule: 80% building, 20% reading.

2.4 Set up a learning loop

A repeatable weekly rhythm beats sporadic deep dives:

- Weekly: ship one small thing (a 30-minute script, a pull request, a blog post).
- Monthly: try one new tool seriously enough to publish a project.
- Yearly: pick one big technology to learn deeply.

2.5 Track your work

A learning-log.md in a personal repo: bullet per week of what you built / learned. After 6 months, you'll have a portfolio.

2.6 Community

Some places to lurk and learn:

- r/learnpython — questions and answers.
- r/Python — projects and news.
- Real Python — articles by topic.
- Python Discord — friendly.
- Local Python meetups — search PyMeetup.

2.7 Job preparation

When you're ready:

- Make sure 3+ projects are on GitHub with READMEs.
- Practice 20 algorithm/data-structure problems on LeetCode / NeoCoderhouse.
- Read career/interview-prep.md in this course.
- Practice explaining your projects in 2 minutes. Practice 5 times.

2.8 Don't burn out

Programming is a marathon. Take days off. The goal is to be doing this in 10 years, not to grind for 6 months and quit.

3. Code Examples

This lesson is reflective — no new code. But here are a few starter snippets for next-step paths:

Path: web backend (FastAPI hello)

```
# pip install fastapi uvicorn
from fastapi import FastAPI
app = FastAPI()

@app.get("/")
```

```
def hello():
    return {"message": "hello"}

# uvicorn main:app --reload
```

Path: data (pandas hello)

```
# pip install pandas
import pandas as pd
df = pd.read_csv("expenses.csv")
print(df.groupby("category")["amount"].sum().sort_values(ascending=False))
```

Path: automation (rich CLI)

```
# pip install rich
from rich.console import Console
console = Console()
console.print("[bold green]hello[/]")
```

4. Common Mistakes

Mistake	What Happens	Fix
Tutorial-hopping	Never finish projects	Build, don't watch
Learning too many tools at once	Surface-only knowledge	Pick one, go deep
Comparing to senior devs	Discouraging	Compare to past-you
Skipping fundamentals refresh	Stale knowledge	Revisit weak areas yearly

5. Practice Tasks

- Pick one specialization path. Write a one-paragraph plan for the next 3 months.
- Open the source of one library you use. Read the first 200 lines of one file.
- Schedule a 30-min weekly slot for "ship something small".

6. Key Takeaways

- Pick ONE specialization path next.
- Build > consume.
- Establish a sustainable weekly rhythm.
- Track your work; community helps.

7. What's Next

You're done with the course content. Finish your capstone, ship it, take the certification test. Then pick your next path and start the next thing.

Good luck — and have fun.

Module 14 — Exercises

These are short scaffolding exercises. The real work of Module 14 is the assignment (Tier A) and project (Tier B).

14.1 Planning

- (E) Write the one-page spec template for your Tier-A capstone choice.
- (M) Break it into 4-6 milestones with time estimates.
- (H) Write a paragraph of explicit non-goals.

14.2 Structure

- (E) Lay out a proj/ package + tests/ folder for your capstone.
- (M) Add a working `__main__.py` with one CLI subcommand.
- (H) Confirm `pip install -e .` and `python -m proj --help` work.

14.3 Testing

- (E) Install `pytest` in your venv; run `pytest` on an empty tests/ folder.
- (M) Write 3 tests for any pure function in your capstone.
- (H) Add a fixture for a temp SQLite DB; test one insert.

14.4 Shipping

- (E) Initialize a git repo for your capstone; first commit.
- (M) Push to GitHub; confirm README renders.
- (H) Tag `v0.1.0` when `v1` works.

14.5 Next steps

- (E) Pick one specialization path; write 1-paragraph plan.
- (M) Star 3 GitHub repos you want to read source of.
- (H) Schedule a recurring 30-min weekly slot for "ship something small".

Module 14 — Quiz

10 MCQs (scaffolding lessons).

Q1

What should you write before opening an editor? A. The first function B. A one-page spec C. The unit tests D. The README

Answer: B (L1)

Q2

What is a "walking skeleton"? A. A class hierarchy diagram B. Smallest end-to-end version touching every layer C. A test plan D. A deployment script

Answer: B (L1)

Q3

Recommended package layout for a capstone: A. One giant file B. Many small files in the root C. A package folder + `__main__.py` + tests/ D. Just functions in `main.py`

Answer: C (L2)

Q4

What runs your code with `python -m mypkg`? A. `mypkg/__init__.py` B. `mypkg/__main__.py` C. `mypkg/cli.py` D. `mypkg/main.py`

Answer: B (L2)

Q5

Which is preferred for new test code? A. unittest B. pytest C. nose D. doctest

Answer: B (L3)

Q6

What does `pytest.fixture` do? A. Auto-installs deps B. Provides setup/teardown for tests C. Skips tests D. Compares outputs

Answer: B (L3)

Q7

What's the most important section of a README? A. License B. Architecture C. Install + Use D. Contributors

Answer: C (L4)

Q8

Before declaring a project "done", you should: A. Add more features B. Run the polish checklist (README, help, tests, .gitignore, version) C. Refactor it from scratch D. Translate it

Answer: B (L4)

Q9

Which is the better long-term learning strategy? A. Watch one tutorial per week B. Ship one small project per week C. Read one book per month D. Memorize the standard library

Answer: B (L5)

Q10

What's a sensible next step after this course? A. Try to master 5 frameworks at once B. Pick ONE specialization path and go deeper C. Switch to a different language D. Stop building, only read

Answer: B (L5)

Module 14 — Tier-A Capstone Assignment

Estimated time: 12-20 hours

Combines: All of Modules 1-13

Brief

Pick one of the Tier-A options below and build it end-to-end: spec, package layout, working code, tests, README, GitHub release.

Option A1 — Personal Finance Tracker

A CLI tool that records, queries, and summarizes personal expenses.

Required features:

- add AMOUNT CATEGORY [--note] [--date]
- list [--category] [--since] [--until] [--limit]
- summary [--month YYYY-MM] — category totals + grand total
- export FILE.csv — write all rows
- import FILE.csv — bulk-load
- SQLite persistence
- Custom exceptions for validation errors
- At least 8 unit tests

Stretch:

- Budgets per category; warn when over.
- A simple matplotlib chart.

Option A2 — News Aggregator

Fetch articles from 1-2 public APIs, store, and serve a CLI dashboard.

Required features:

- Pull from at least one API (use the GitHub API, JSONPlaceholder, or <https://newsapi.org/> if you sign up for the free tier — adjust based on availability).
- Persist articles in SQLite.
- fetch — pull new articles into the DB.
- list [--source] [--since] — show stored articles.
- search QUERY — full-text-ish search on title/body.
- mark-read ID.
- Custom exceptions for API and DB errors.
- At least 8 unit tests.

Stretch:

- Daily summary email (using smtplib) or a generated Markdown digest.

Option A3 — Habit Tracker

Track daily habits, streaks, weekly reports.

Required features:

- add-habit NAME
- check-in NAME [--date YYYY-MM-DD]
- today — show which habits are checked in / pending today.
- streak NAME — current streak length.
- report [--weeks N] — completion percentages.
- SQLite persistence.
- Custom exceptions.
- At least 8 unit tests.

Stretch:

- Reminder system that lists pending habits if any are missed today.

Shared Constraints (all options)

- Package layout per Module 14 Lesson 2.
- pip install -e . works.
- python -m yourpkg --help shows useful help.
- pyproject.toml with name, version, deps.
- README.md with Install / Use / Demo sections.
- .gitignore excludes secrets, DB files, .venv.
- All SQL parameterized.
- requests for HTTP (if used) with timeouts.
- Pushed to GitHub, tagged v0.1.0 when done.

Grading Rubric

Criterion	Weight
All required features	35%
Code organization	15%
Tests pass and cover meaningfully	15%
README + ship checklist	15%
Style, exceptions	10%
Walkthrough explanation	10%

Submission

A GitHub repo with:

- The package

- tests/ folder
- README.md
- LICENSE
- A short (3-5 min) screen recording or written walkthrough explaining the design.

Module 14 — Tier-B Mini-Capstone Project

Estimated time: 4-8 hours

Combines: Selected concepts from Modules 1-13.

Pick one of the smaller projects below. The goal is a tightly-scoped, polished tool.

Option B1 — Markdown Table-of-Contents Generator

Read a Markdown file and inject (or print) a table-of-contents linking each heading.

Features:

- `python -m mdtoc input.md` — prints the TOC.
- `--inject` — replaces a `<!-- TOC -->` marker in the file.
- Supports `#` through `#####`.
- Generates GitHub-style anchor links (lowercase-with-dashes).

Concepts: strings (M6), regex (M6 L4), pathlib (M7 L5), packaging (M10).

Option B2 — Bulk File Renamer

Rename files in a folder by a pattern, with a dry-run mode.

Features:

- `python -m renamer PATTERN REPLACEMENT [DIR]` — preview rename plan (dry-run by default).
- `--apply` actually performs the renames.
- `--regex` switches PATTERN to a regex.
- Reports collisions before renaming.

Concepts: pathlib (M7 L5), regex (M6 L4), argparse (M10), error handling (M8).

Option B3 — Wikipedia Summary Bot

CLI that takes a query and returns a 2-paragraph summary from Wikipedia's REST API.

Features:

- `python -m wikibot QUERY`
- Uses Wikipedia's `/api/rest_v1/page/summary/{title}` endpoint.
- Handles 404 (no article), 429 (rate limit), timeouts.
- `--lang` chooses language (default en).

Concepts: requests (M12), JSON (M12 L3), errors/timeouts (M12 L5), packaging (M10).

Shared Constraints

- Package layout per Module 14 Lesson 2.
- `python -m yourpkg --help` works.
- README with Install / Use sections.
- At least 3 tests for any pure logic.
- Pushed to GitHub.

Grading Rubric

Criterion	Weight
All listed features	40%
Layout + packaging	20%
README + help text	15%
Tests (≥ 3 meaningful)	15%
Style	10%

Submission

A GitHub repo, same standards as the Tier-A capstone.

After you finish your Tier-B project: take the certification test and check the career guide.